



A machine learning approach for performance-oriented decision support in service-oriented architectures

Tehreem Masood, et al. *[full author details at the end of the article]*

Received: 9 March 2020 / Revised: 2 July 2020 / Accepted: 23 August 2020 /

Published online: 23 September 2020

© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract

Enterprise IT performance can be improved by providing reactive and predictive monitoring tools that anticipate problem detection. It requires advanced approaches for creating more agile, adaptable, sustainable and intelligent information systems. Service-oriented architecture (SOA) has been used in significant performance-based approaches by information system practitioners. Organizations are interested in performance-based decision support along the layers of SOA to maintain their sustainability for service reuse. Reusability is a very important aspect of Service-based systems (SBS) to analyze service or process reuse. This helps in achieving business agility to meet changing marketplace needs. However currently, there are many challenges pertaining to the complexities of service reuse evolution along SBS. These challenges are related to the sustainability of service behavior during its lifecycle and the limitations of existing monitoring tools. There is a need for a consolidated classified knowledge-based performance profile, analytical assessment, prediction and recommendation. Therefore, this paper provides a semantic performance-oriented decision support system (SPODSS) for SOA. SPODSS provides recommendations for suggesting service reuse during its evolution. SPODSS is supported by five building blocks. These blocks are data, semantic, traces, machine learning, and decision. SPODSS classifies data, validates (analytical assessment, traces, semantic enrichment) at different time intervals and increased consumption and prediction based on consolidated results. It handles the dynamic evolution of SBS and new or changed user requirements by ontology development. Finally, SPODSS generates recommendations for atomic service, composite service, and resource allocation provisioning. To motivate this approach, we illustrate the implementation of the proposed algorithms for all the five blocks by a business process use case and public data set repositories of shared services. Sustainability and adaptability of service-based systems are ensured by handling new business requirements, dynamicity issues and ensuring performance. Performance criterion includes functional suitability, time behavior, resource utilization, and reliability in terms of availability, maturity, and risk.

Keywords Web service · Service related knowledge · Service ontology · Quality of service · Service reuse · Data mining · Inference rules

1 Introduction

Service-oriented architecture (SOA) is emerging as a powerful paradigm for organizations that need to integrate their applications within and across organizational boundaries (Arsanjani et al. 2007a). A system following this architecture is composed of several services. These services can be combined as building blocks for the composition of larger systems, known as Service-Based Systems (Kyusakov et al. 2012). SBS rely on service providers to ensure that their services comply with the agreed Quality of Service (QoS) (Sachan et al. 2014). Today's software systems are becoming increasingly integrated into the lives of their end-users and their ever-changing environments and needs. These demands lead to the growing complexity of systems. Microservices emerge as new architectural style that relies on small, decentralized, and autonomous services that work together (Tizzei et al. 2017). Microservices and SOA share a large set of architectural principles and solutions in the general space of SBS (Bogner et al. 2018). The microservice architecture is a variation of a Service-Oriented Architecture (SOA) that allows a service to be broken down into a number of smaller but independently concurrent running units so that both performance and maintainability of the application can get a big improvement (Wang and Fahmi 2018). The advancements in modern technology and the continuously changing requirements caused by dynamic business environments incur new challenges for engineering and provisioning service-based applications (SBAs) (Mirandola et al. 2014). SBAs must become drastically more flexible. They should be able to operate and evolve in highly dynamic environments. They require up-to-date QoS information for their proper management at the different lifecycle stages, from their initial construction until their end. A variety of approaches and techniques addressing different forms of monitoring and adaptation have been proposed today (Yoon et al. 2016; Oriol et al. 2015). Nevertheless, these results are still fragmented. These existing works do not consider the variability of QoS parameters. In the perspective of reusability to lower the development cost and quickly provide values to enterprises, the center Blackduck for open source research and innovation releases in 2017 an open source 360-degree survey. The center collect answers from 800 IT practitioners and cross-domain end-users. The survey points the need for searching, evaluating, retrieving technical components to reuse, and considering uncertainty on the quality. These requirements are confirmed in (Paschali et al. 2017). SOA performance relies on the set of parameters that we can define and monitor their evolution in each specific deployment environment (on-premises, public cloud, private cloud, etc.). Service performance can depend on the structure of the service interface, the XML parsing to route a message to a service handling, messages data persistence, messages data transformation, etc. Performance can be measured, analysed, and guaranteed by the qualitative and the quantitative characteristics provided by ISO 25010. Some important aspects for SOA based organizations are to monitor performance along different time dimensions and SOA layers, to determine the level of maturity, to avoid risks or threats and provide efficient decision support ensuring sustainability and adaptability of SBS considering governance rules. Advances in computer technology are dynamic, and they impact information systems including Decision support systems (DSS) (Adam 2008). This phenomenon highlights that information systems will evolve accordingly in various directions. Existing works highlight the concepts of data mining and ontologies to provide efficient decision support applications (Fahad and Qadir 2008). In machine learning and statistics, classification algorithms identify a set of new observations based on a training set of data that helps in decision making. Ontologies allow tracking and exposing performance and decision concepts in a structured form and evolve with new or

changing business needs. It is difficult to compose dynamically several performance indicators and no analytical assessment with time at the different SOA layers to ensure information system sustainability. Moreover, estimation of the impact of new consumption of services and handling governance problems have remained as challenges. However, an intelligent information system requires intelligent DSS to cover above mentioned challenges. Therefore, this research aims to create a support system for accelerating the monitoring of Web services and intelligent decision making for their reuse. The above-stated research problem leads to the following research questions to guarantee efficient performance-based decision support for service reuse:

- RQ1. Are technical indicators (e.g. ServiceTime, ResponseTime, FailureRate, etc.) compliant with new quality standards?
- RQ2. How to provide analytical assessment for performance evolution (via atomic services, services chaining, services orchestration, services consumption in business tasks, etc.) with time dimension and evaluate the impact of increased consumption of services?
- RQ3. How to incorporate changing or new End User service performance objectives with the evolution of the service life cycle?
- RQ4. How to handle the resource demands of the system at different workloads?
- RQ5. How to provide efficient performance-based decision support for service reuse?

It is important to monitor and examine the performance of SOA layers with respect to a more consistent quality model implementing connected assessment and decision concepts as answers to the previous research questions. There are fewer concrete data on service reuse in current SOA engagements. Rather, a lot of recent writings have just pointed out the challenges in achieving service reuse. The main contribution in this paper is to provide analytical and predictive assessment and recommendations for service reuse during its evolution while preserving acceptable performance, maturity and risk levels. We present the list of contributions in this paper hereafter:

- C1: Maximizing performance of SBSs by adding new quality characteristics with reference to the latest quality standards.
- C2: Semantic Performance Profile: Performance profile will provide insights about performance-oriented decision support concepts and technical indicators.
- C3: Classification of Knowledge-based data representations of service performance linked to previous consumptions to build new reuse decision model.
- C4: Machine learning for generating efficient and optimized decisions by increasing the consumption of data.
- C5: Ontology revision for the generation of inference rules to get a global decision for service reuse.
- C6: Recommendations to reuse atomic and composite service, dynamic QoS provisioning and upgrading governance policies if required.
- C7: Definition of a use case where a performance-based decision support system is applied to validate and measure. In this way, sustainability, reusability and adaptability of SBS are ensured.

The remaining of the paper is organized as follows. Section 2 (see Sect. 2) reviewed related work on service reuse solutions, performance monitoring, risk management, maturity models

and ontologies. Performance-based decision support framework is proposed in Section 3 (see Sect. 3). Experimental results are depicted in Section 4 (see Sect. 4). Section 5 (see Sect. 5) presents the conclusion of the paper with future research directions.

2 Literature review

In this section, we analyse existing relevant research works. We focus on six areas namely service-reuse solutions, performance monitoring, risk management, maturity models and ontologies.

2.1 Systematic analysis

The review is carried out using Scopus, Web of Science and ISI Web of knowledge with different search criteria. Our first search criterion is service reuse. For that, we extracted 711 research papers from the year 1991 to 2019. Most of the papers under this category are not related to service-based systems but rather focus on computer networks or some other domains like water and energy consumption. Therefore, we selected papers that are relevant to service-based systems that result in 15 papers. The second search criterion is service performance based on quantitative indicators. For that, we extracted 3103 research papers from the year 2004 to 2019. We have selected 2842 papers relevant to the computer science domain in the second analysis. 994 papers are related to SOA that are selected in the third analysis. The fourth analysis resulted in 486 papers based on Web services. The fifth analysis included 97 papers related to the monitoring of Web services. We have eliminated papers published before the year 2010 because of their application in the field of computer networks which resulted in 24 papers in the final analysis. We categorized these papers based on performance monitoring. The third search criterion is qualitative indicators such as risk and maturity. From this analysis, we select 14 papers for risk management while 25 papers have been selected based on the existing maturity models. The fourth search criterion is service performance storage and decision support. For that, we extracted 103 research papers from the year 2000 to 2019 based on ontology structuring and reasoning. We select 15 papers that are relevant to the quality of service ontologies and among them, few papers highlight decision support based on ontologies.

2.2 Service reuse solutions

Feuerlicht et al. (2007) propose principles for service reuse. Perepletchikov et al. (2007) propose an approach that measures cohesion of service. Xue et al. (2018) propose a technique of process partitioning. Khoshkbarforousha et al. (2010) propose an approach for evaluating the reuse of composite services. Choi and Kim (2008) present a model for the reuse of atomic services in SOA. Doultsinou et al. (2009) provide a procedure for selecting requirements and mapping with service reuse. Lee et al. (2010) provides a feature-oriented approach, to analyze and identify reusable services. Ahmed-Kristensen and Vianello (2015) propose a model named as reuse service knowledge (RSK). Allen et al. (2016) proposes a method that highlights the reuse of communication services. Sangaiah et al. (2019) propose quality of service-oriented composition to maximize service reuse. We identify two important areas for service reuse: service compliance, and service performance monitoring. It is important to align performance

metrics with business goals such as the increase in flexibility and reduction of business process costs. Existing metrics have not been pragmatically validated.

2.3 Performance monitoring

Web services can be monitored by evaluating their performance. This performance can be monitored along the different layers of SOA Reference Architecture (RA) (Arsanjani et al. 2007b) as mentioned in the introduction section (see Sect. 1). This architecture is structured in nine layers comprising an operational systems layer, an enterprise components layer, a services layer, a business processes layer, an integration layer, an information architecture layer, Qos layer, and a governance layer. Therefore, we discuss the existing works related to these layers based on the quantitative indicators in forthcoming parts. Oriol et al. (2010) propose a tool SALMon to assesses the satisfaction of SLA. Valls et al. (2012) propose iLAND for the timely reconfiguration of real-time systems for services. Asadollah and Chiew (2011) develop a monitoring method for web services response time. de Gyves Avila and Djemame (2013) propose an optimization model that performs service selection. Valls and Val (2013) propose reconfiguration of service compositions for distributed real-time systems. Zheng et al. (2012) investigate the quality of service of real-world web services and prediction of performance. Kahlon et al. (2014) propose a technique based on web services monitoring life cycle processes. McKee et al. (2015) present a probabilistic modeling approach to capture the relationship between service performance and the underlying environment. Boumahdi et al. (2016) propose a model name as SOA + d that deal with business process performance. He et al. (2016) present a spectrum-based approach to quickly localize runtime anomalies. Aschoff and Zisman (2011) propose proactive adaptation of service composition. Fan (2013) proposes a method to personalize composite service. Sheng et al. (2014) provide an analysis of web service composition and technologies. Tari et al. (2011a) propose a method to benchmark different SOAP bindings in wireless environments. Tari et al. (2011b) propose a similarity-based SOAP multicast protocol (SMP) to reduce the network load. Existing approaches do not provide the decisional aspect for SBS to cover the service reuse capability. Approaches are primarily static in nature, which makes them unsuitable for assuring runtime and emerging system qualities. Distribution of the quality models along the time dimension and the identification of their relationships are missing.

2.4 Risk management

Risk management plays an important role in SBS to identify the risk, the impact of risk and mitigating actions for risk. Sackmann (2008) proposes a model as an IT risk reference model for the business process-oriented view. Sackmann et al. (2009) extends his work and expresses these relations in a matrix-based description. Sadiq et al. (2007) propose a method for business process compliance. Weber et al. (2008) describes a method for validating whether the states reached by a process are compliant with a set of constraints. Jakoubi et al. (2010) propose a method as risk-oriented process evaluation (ROPE). Based on the ROPE methodology, Tjoa et al. (2010) and Suriadi et al. (2014) propose a reference model for risk-aware evaluation. Milanovic et al. (2008) propose analytical assessment method for demonstrating availability related to services, underlying ICT infrastructure, and human resources.

2.5 Maturity models

A maturity model is a way to assess and improve business processes constantly. It describes the typical patterns in the evolutionary process of technology and business development of an enterprise (Solli-Sæther and Gottschalk 2010). It is used to rate the capabilities of maturity elements and to select the processes for improving their maturity (Kohlegger et al. 2009). Röglinger et al. (2012) provide an analysis of a set of BPMMs. De Bruin and Doebeli (2015) develop a holistic BPM maturity model. BPMM-OMG (B OMG 2008) and vPMM (Lee et al. 2009) provide a maturity model and the assessment model. Rohloff (2009) provides PMMA as a holistic evaluation of all areas relevant to BPM based on a complete set of criteria. Krivograd et al. (2014) define an intelligent maturity model tool. The Capability Maturity Model Integration (CMMI) models are the most well known generic maturity models. The CMMI Product Suite has three models that are CMMI-DEV, CMMI-SVC, and CMMI-ACQ for development, services, and acquisition, respectively (CMMI Product Team 2010). SCAMPI (SCAMPI Upgrade Team 2011) is a standard CMMI appraisal method for process improvement. Hirschheim et al. (2010) publishes a study report. Later, it has been adopted by The Open Group (2011). Hensle and Deb of Oracle Corporation (2008) highlight SOAMM as a guide to accelerate the enterprise SOA adoption. Baghdadi (2014) proposes the SOA maturity framework to guide the SOA adoption process. There are several SOAMM contributed by both academia and industry such as IBM SIMM/OSIMM, HP SOAMM (Pugsley 2006), Oracle SOAMM, iSOAMM (Rathfelder and Groenda 2008). Pulparambil and Baghdadi (2015) propose a set of overlapping categories between CMMI and SOAMM. Later provide a framework for comparison of SOA maturity models (Rohloff 2009). After mapping CMMI process areas and SOAMM dimensions, Pulparambil et al. (2017) propose an approach based on SOA adoption for maturity levels. Bani-Ismael and Baghdadi (2016) and Pulparambil in (2019) propose identification methods as SOA maturity models to guide service selection. Types of risks with respect to SBS have not been mentioned in the above-studied research papers. Along with that, the impact of risk is also missing. There is also a lack of prescriptive properties in terms of mitigation actions for risks and maturity. The evolution from current maturity level i to next maturity level $i + 1$ for SBSs is not handled. We now discuss the existing approaches based on QoS and decision-based ontologies in the next section 2.6 (see Sect. 2.6).

2.6 QoS ontologies

As explained in the introduction, QoS is a non-functional property of service. QoS attributes are stored in ontologies in general due to several advantages (Zagorulko and Zagorulko 2010; Tang and Meersman 2008). The first advantage is the modeling and structuring of performance knowledge related to SBS. The second most important advantage is to infer new knowledge by reasoning on ontologies (Gorogiannis et al. 2009). The third very important advantage is the dynamic nature of ontologies. In this part, we provide the analysis of various QoS ontologies that exist in the literature. Giallonardo and Zimeo (2007) propose an ontology named as on QoS defined with OWL language. Lin et al. (2008) proposed ontology-based QoS-Aware support for semantic Web services. D'Mello and Ananthanarayana (2009) propose an extension to the OWL-S to support the storing of QoS and business offerings. Benaboud et al. (2012) propose a semantic Web service discovery approach based on agents and ontologies. Moraes et al. (2008) propose an ontology named MonONTO. Pakari et al. (2014) propose a

hybrid semantic matching approach for service discovery from OWL-S ontology. Chhun et al. (2016) propose Web service ontology (WSOnto). Some of the works focus on providing conceptual modeling to infer new knowledge (Benomrane and Ayed 2014) while, some other work discusses the exploitation of results from ontologies and interactions via ontologies [?]. Above mentioned QoS ontologies are not adaptable to new or changed End User requirements as the reuseperspective of services (atomic reuse, chaining, orchestration, etc.) comparing to the inferred recommendation or to a restriction in a specific performance criterion (response time, failure rate, risk, maturity, etc.). Moreover, QoS ontologies do not evolve with time. Since several QoS information is stored in the above-mentioned literature, there is a need for composite semantic ontology.

2.7 Critical analysis

From the analysis of the proposed literature review, we assume that service reuse relies on service compliance, and service performance monitoring of some parameters at the different layers of the SOA Reference Architecture. The SOA maturity model covers additional parameters to assess services lifecycle and guide the SOA adoption process. The assessment of these parameters is very subjective and there is a lack of prescriptive properties in terms of mitigation actions for risks and maturity. QoS ontologies are proposed as evolutive container for modelling, structuring service performance parameters, reuse rules, and finally the possibility to infer new rules. All of these SOA related concepts can serve better the service reuse perspective when orchestrated together to connect end-users' requirements (atomic service reuse, service orchestration, etc.) with a semantic representation of service performance monitoring, service risk mitigation and SOA maturity concepts.

3 Proposed semantic performance-oriented decision support system

The related work analysis highlight the need of a fully automatic performance-based system for service reuse in the dynamic SOA based organization, where efficient decisions are required to be made on-the-fly. Our goal is to ensure adaptability, reusability, agility, dynamicity, and sustainability of SOA based information systems. For this purpose, we propose a semantic performance-oriented decision support system (SPODSS). SPODSS is shown in Fig. 1. SPODSS is composed of six building blocks. These blocks are data handler, ontology builder, reasoner, classifier, traces handler and decision generator. We explain the implementation of each block step by step in forthcoming parts.

3.1 Data handler

The data handler manages both quantitative and qualitative technical indicators. Performance can be measured, analyzed and guaranteed by qualitative and quantitative characteristics provided by ISO 25010 (ISO 2011). These characteristics are performance efficiency and reliability. Performance efficiency is analyzed based on sub characteristics, time behavior, resource utilization, and capacity. Reliability is analyzed based on availability, maturity and fault tolerance in terms of risk. The first step of this component is performance monitoring. This performance monitoring is performed based on the quantitative technical indicators. The second step is maturity evaluation. Maturity is evaluated by using SOAMM levels of

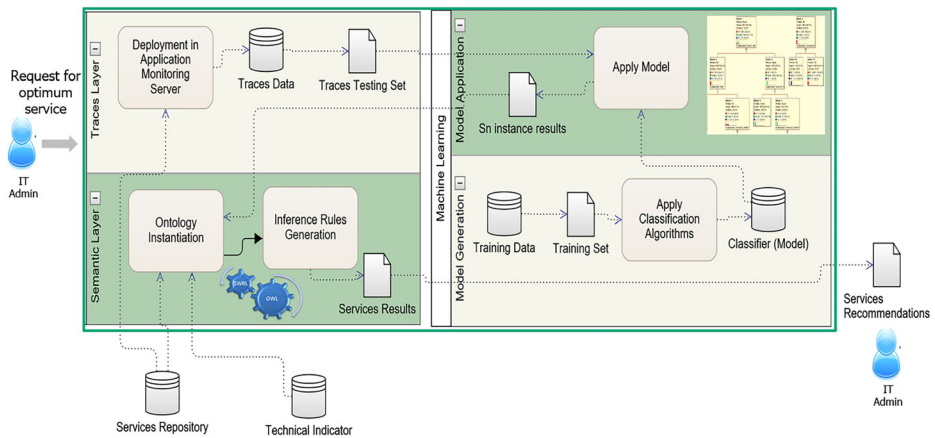


Fig. 1 Proposed semantic performance-oriented decision support system (SPODSS)

Pulparambil, et al. We evaluate maturity based on the service instance time and availability of a service or a process using SOAMM levels. The last step is based on risk analysis. Risk is analyzed by identifying risk, measuring the level of risk, calculating the probability percentage, risk gravity, making the assessment and mitigating action.

3.2 Ontology builder

The ontology builder is responsible for the dynamic performance profile and the results generation. It instantiates ontology in several iterations. These iterations involve storing and instantiating ontology for the concepts of services, services instances, performance indicators, traces, performance indicators values for services instances and decision. In this way, we reuse ontological concepts in multiple iterations that we will explain in section 4 (see Sect. 4) for the implementation of the use case. We develop ontology in Protégé Musen (2015) and used DL Reasoner to verify their consistency. Service network ontology is already published (Masood et al. 2015). The ontology builder generates a composite ontology.

3.2.1 PODSOnt description

In this part, we explain our proposed ontology that integrates all performance ontologies at SOA layers at first level, extends ontologies at SOA layers and extends service layers in terms of performance and decision. As a result, a composite and coherent global ontology in terms of performance and decision support for SBS is produced. This ontology is named as performance-oriented decision support ontology (PODSONt) and is developed to maximize the service reuse capabilities. Figure 2 provides the structure of PODSONt. PODSONt is proposed as an extension of the one proposed by Chhun et al. (2016), identified as the most generic ontology covering functional and non-functional performance concepts. PODSONt starts with the service domain concept. We have already proposed and published ontologies for performance-oriented layers such as service, business process, integration and resource

layers (Masood et al. 2016a; Masood et al. 2016b). Now, we highlight performance and decision concepts in this ontology. The service layer has service 1 which has QoS concepts. QoS is defined by performance efficiency, maturity, risk and recoverability concepts. Performance efficiency is measured by time behavior, resource utilization and capacity. Maturity is measured at five levels. Level 0 and level 1 are measured by service instance time while the other levels are measured by availability. Risk is measured by risk gravity, and risk gravity is measured by risk probability and severity. Recoverability has a failure rate and MTTR. MTTR is measured by service downtime and time intervals. The failure rate is measured by service instance response count and total service instance response count. Service has a decision concept. The decision is measured by service instance ID and total service instance ID. The decision has result defined by class OK, class Maybe, and class NO. If the result is OK then it means that service is recommended for reuse else if the result is Maybe, service may be recommended and finally, if the result is NO then service is not recommended. By the implementation of ontological concepts, properties, and relationships, new knowledge can be deduced by implementing inference rules. The evolving or dynamic nature of PODSOnt permits to assimilate or cumulate new concepts. As a result of this evolution in the form of several iterations of PODSOnt concepts discussed above, inference rules also evolve. The implementation of the inference rules is presented in the following section (see Sect. 3.3).

3.3 Reasoner

Reasoner is responsible for the creation of consolidated knowledge base inference rules. Based on the PODSOnt concepts, reasoner generates inference rules in two iterations. At first, it generates rules based on performance while at the second level,

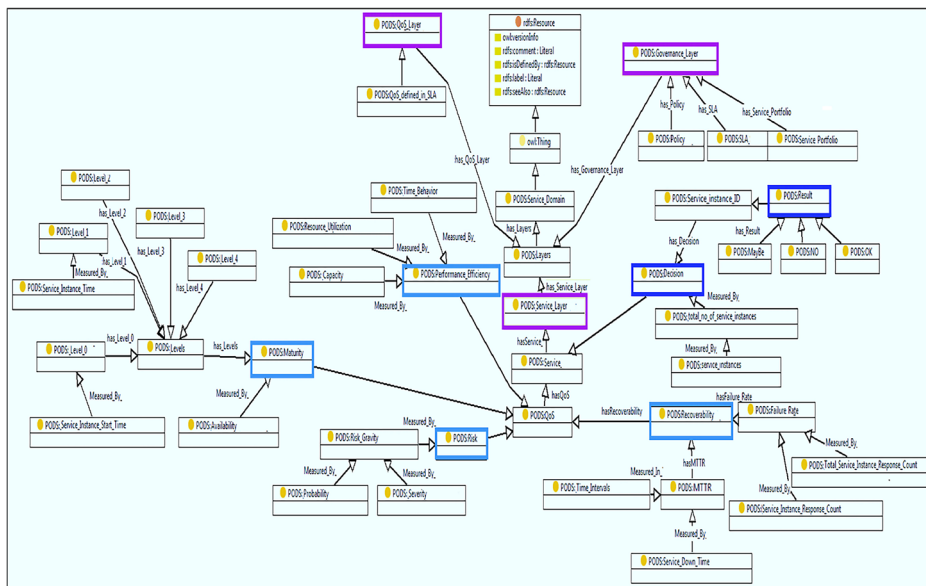


Fig. 2 Performance-oriented decision support ontology (PODSOnt)

it generates decision-based rules. Performance-oriented inference rules that compose the knowledge organization from the relationships between PODSOnt concepts are shown in Fig. 3 while decision-oriented inference rules are illustrated in Figure 4. The right column contains the rules corresponding to the technical indicators listed in the left one. We develop inference rules in SWRL using Protégè. We have already developed and published some inference rules (Masood et al. 2016c). The implementation of proposed inference rules using PODSOnt concepts, properties and relationships provide a knowledge base. Since inference rules can evolve with time, they provide enhanced analytic and manipulations to help decision makers, making the right decisions accurately on the fly.

3.4 Classifier

Data is classified into two steps. The first step is model generation while the second step is model application. In the first step of model generation, training data is gathered from the database to create a training set. This training set is used to generate a model by applying classification algorithms (logistic regression, naïve Bayes, support vector machine, and Decision Trees). As a result, the validation set is provided to the model application step. Model application step takes the optimum resulted model and applies on the testing set of traces generated by traces handler block.

3.5 Traces handler

Traces handler involves the deployment of services and processes in the application server to monitor their performance. It provide analytics of technical indicators and status report of services and processes at run time in different time dimensions. As a result, it stores traces in the database. Traces are stored in the database during different time stamps. These data help the framework to analyze performance degradation's at run time. As a result, a testing set of traces has been generated and the

Decision Variables	SWRL
Count_Service_Instance_ID_Recommendation_OK	Service_Instance_ID (?p) ^ hasResult(?p, ?OK) -> switcount(?OK)
Count_Service_Instance_ID_Recommendation_MayBe	Service_Instance_ID (?p) ^ hasResult(?p, ?MayBe) -> swrl:count(?MayBe)
Count_Service_Instance_ID_Recommendation_NO	Service_Instance_ID (?p) ^ hasResult(?p, ?NO) -> sval:count(?NO)
Set_Service_Recommendation	swrl:greater(Count_Service_Recommendation_NO, (Count_Service_Recommendation_OK ^ Count_Service_Recommendation_MayBe),?x) -> Set_Recommendation (?NO)
Set_Service_Recommendation	swrl:greater(Count_Service_Recommendation_OK, (Count_Service_Recommendation_NO ^ Count_Service_Recommendation_MayBe),?x) -> Set_Recommendation (?OK)
Set_Service_Recommendation	swrl:greater(Count_Service_Recommendation_MayBe, (Count_Service_Recommendation_OK ^ Count_Service_Recommendation_NO),?x) -> Set_Recommendation (?MayBe)

Fig. 3 Performance-oriented inference rules

Technical Indicators	SWRL Rules
Availability	swrl:divide(Total_Response_Count,Total_Request_Count*100),?x) -> Availability (?x)
Throughput	swrl:divide(no_of_active_request,Average_Response_Time,?x) -> Throughput (?x)
Delay	swrl:difference(Service_Deployed_Time,Service_Up_Time,?x) -> Delay (?x)
Response_Time	swrl:difference(Service_End_Time,Service_Start_Time,?x) -> Response_Time (?x)
Bandwidth	swrl:divide(service_instance,Time,?x) -> Bandwidth (?x)
Failure_Rate	swrl:difference(Request_Count,Response_Count,?x) -> Failure_Rate (?x)
MTTR	swrl:divide(Service_Down_Time,No_Intervals,?x) -> IVITTR (?x)
Service_Time	swrl:sum(Service_Start_Time,Service_End_Time,?x)-> Service_Time (?x)
Available_Memory	swrl:difference(Memory_Allocated,Memory_Usage,?x) -> Available_Memory (?x)
Risk_Gravity	swrl:multiply(Rrobabikty,Severity,?x) -> Risk_Gravity (?x)
Maturity_level_0	swrl:equal(Service_Instance_Time,0,?x) -> Maturity_Level0 (?x)
Maturity_level_1	swrl:greater(Service_Instance_Time,0.00000001,?x) -> Maturity_Level1 (?x)
Maturity_level_2	swrl:greater(Availability, 98.999,?x) -> Maturity_Level2 (?x)
Maturity_level_3	swrl:greater(Availability, 99.99,?x) -> Maturity_Level3 (?x)
Maturity_level_4	swrl:equal(Availability, 100,?x) -> Maturity_Level4 (?x)
Total_No_of_Service_Instances	swrl:sum(Service_Instance_ID,?x) -> Total_Serviceinstances (?x)

Fig. 4 Decision oriented inference rules

model is applied to it. We have already published the implementation aspects of the traces handler (Masood et al. 2018).

3.6 Decision generator

The model application step of the classifier block generates results for services instances. These results are passed to the semantic block where PODSont is instantiated to make decisions based on each service. As a result, the decision generator generates results by instantiating PODSont and decision-oriented inference rules. It takes validation set from classifier, traces testing set from traces handler and generates results. Finally, recommendations have been made to suggest reuse atomic service, composite service, resource allocation provisioning and upgrading governance policies if required. We now explain the experimental results and discussion.

4 Experimental results and discussion

In this section, we evaluate SPODSS by fetching services from a public repository of shared services and usage scenarios.

4.1 Data set

To validate the proposed concepts and implementation details described in the previous sections (see Sect. 3), we gather QoS web services inputs from two public repositories (Dream 2011).

- Repository 1: This dataset describes real-world QoS evaluation results from 339 users on 5825 Web services.
- Repository 2: This dataset describes real-world QoS evaluation results from 142 users on 4500 Web services over 64 different time slices (at 15-min intervals).

The inputs provided from both repositories are analyzed, aggregated and classified. Only 100 instances (with all necessary performance criteria) for about 1000 services are elected (based on the completeness of the QoS data) for the definition of our classification model. The recommendations are provided based on the rules sets provided in section 3 (see Sect. 3). This rules set is related to performance efficiency, reliability, maturity, and risk. The proposed validation was conducted in an agile and incremental approach. On the one hand, the ontological models are instantiated for the raw data as well as for the tracesTM results. On the other hand, data mining techniques have been applied to generate first level results. Final results involve the evolution of ontologies and inference rules with the analytical results for service reuse.

4.2 Process

First, raw data of service instances from the existing repository of shared services are instantiated in the proposed ontologies and inference rules. After this, classification models have been applied. From these results, we chose the model that generates the most optimum results. Secondly, we deploy the application in the performance monitoring server, generates traces and apply the selected model. We consolidate both results and evaluate final consolidated results for each service from all services instances results. PODSOnt is instantiated to generate aggregation rules for each service. Finally, we get the results for each service based on which we provide decisions to the End User in terms of atomic or composite service reuse recommendation. We now explain the data analytics generated at each level of use case implementation.

4.3 Service reuse use case

We need to develop a business process in order to analyze the collaboration of two business partners that are partner one and partner two. For this purpose, we explain the use case of the business process and the underlying services that we need to develop. This use case is used to evaluate the capacity of these two partners to industrialize products from common customer projects. Since the application is supported by different service tasks, it is very time-consuming to follow the traditional process of development. It incurs cost, time, resources and trust problems. Therefore, from the perspective of maximizing service reuse, we identify some interesting APIs to reuse in the FIWARE industry portal (Fiware technologies enabling industry 4.0 2015).

4.4 Data mining, traces, and semantic analytics

It involves the application of a machine learning approach. We chose classification algorithms of machine learning approach to analyze the generated semantic rules and traces to get the performance evaluation results for services. Tests are performed in two iterations.

4.4.1 Iteration 1

In this iteration, analytics are generated by applying classification algorithms (Logistic regression, Decision Tree, Naïve Bayes and Support Vector Machine). We designate 50% of data coming from the repository for this iteration. We classify this data by applying classification algorithms and get analytical results. After the formulation of performance criteria, we construct a model based on classification algorithms. For this purpose, we define rules to evaluate the recommendation weights such as OK, NO and MayBe. Basic rules to generate recommendations weight are shown in Fig. 5.

We collect and consolidate these data from three SLA experts from three different industrial companies. After the assignment of all weights, the remaining monitoring data is updated to the recommendation weight NO. The other recommendation weights are OK and MayBe. Recommendation weight is assigned to OK if the maturity is greater or equal to 3 OR maturity is greater or equal to 2 AND risk is less than or equal to 0.0001 AND available memory is greater then and equal to 0.8 AND failure rate is less then and equal to 0.03. Another predicate is used to assign the recommendation weight as OK that is if maturity is greater then and equal to 2 AND risk is less then and equal to 0.001 AND available memory is greater then and equal to 0.5 AND failure rate is less then and equal to 0.05. Recommendation weight is assigned as MayBe if maturity is greater then and equal to 2 AND service time is less then and equal to 0.5 AND failure rate is less then and equal to 0.05. After applying these rules for all algorithms, we get the analytics. After the construction of analytical results from all models, we evaluate them based on some parameters. The parameters that we chose to evaluate classification algorithms are predictive confidence and precision. Classification algorithms' results based on parameters are shown in Table 1. These results show that the decision tree algorithm produced the most optimum results with a precision percentage of 99,8787 and predictive confidence of 99,7247 in this particular case.

```

Update MonitoringData
set Recommendation = 'NO'

Update MonitoringData
set Recommendation = 'OK'
where
Maturity >=3 OR (Maturity >=2 and Risk <=0.0001 and Available_Memory >=0.8 and
Failure_Rate <= 0.03)

Update MonitoringData
set Recommendation = 'OK'
where
Maturity >=2 and and Risk <=0.001 and Available_Memory >=0.5 and Failure_Rate <= 0.05

Update MonitoringData
set Recommendation = 'MayBe'
where
Maturity >=2 and Service_Time <= 0.5 and Failure_Rate <= 0.05

```

Fig. 5 Basic rules for the assignment of recommendation weights

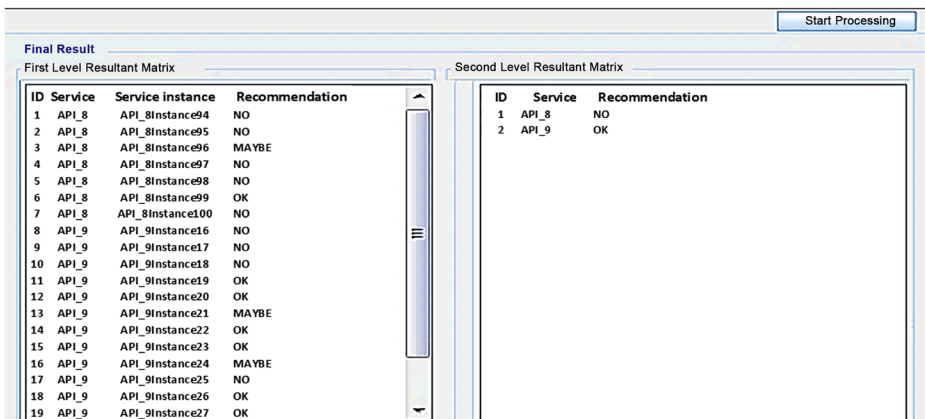
Table 1 Classification algorithms results

Algo_Name	% Predictive Confidence	% Overall Accuracy	% Average Precision
CLAS_DT_1_7	99,7247	99,8787	99,8165
CLAS_NB_1_7	94,7271	95,0156	96,4847
CLAS_SVM_1_7	84,4374	85,1296	89,6249

4.4.2 Iteration 2

Iteration 2 involves the deployment of an existing application from the FIWARE portal in the performance monitoring server and gathered traces from it. Figure 6 illustrates Get service of 3DScan application information in the application server. We perform the first test to evaluate the performance of getting service. The upper part of the figure provides the information and statistics of the operation while the lower part displays the KPI analysis graph. This test is performed to evaluate the performance efficiency in terms of time-frequency and faults. For this purpose, three KPI's and three metrics are analyzed. The three KPI's are request count, response time, and fault count while three metrics are maximum response time, minimum response time and average response time. KPI analysis graph shows the average response time of the service at different time intervals. The values of average response time and time instance correspond to the unit of milliseconds. The analysis shows that no fault is encountered during this test.

The analytical result of the model generation is shown in Fig. 7. This analytical result shows service ID, service instance ID, AP ID, values of technical indicators and recommendation. After this, the selected optimum decision tree algorithm is applied to the traces gathered from the server. Finally, it consolidates both results generated from the model application on traces of the server as well as the results generated in iteration 1. The resultant matrix from the decision tree model is shown in Table 2. The first column of the figure shows the target weights. These target weights are MAYBE, NO and OK. The second column presents the MAYBE value for each target weight while the third column shows NO value for each target weight. Finally, the last column displays the OK value for each target weight. From this result, we can see that there are 0 instances for the class of MayBe with MayBe. 2.6465 instances result for the class of MayBe with NO and the same number of instances justified the class of



First Level Resultant Matrix		
ID	Service	Service instance
1	API_8	API_8Instance94
2	API_8	API_8Instance95
3	API_8	API_8Instance96
4	API_8	API_8Instance97
5	API_8	API_8Instance98
6	API_8	API_8Instance99
7	API_8	API_8Instance100
8	API_9	API_9Instance16
9	API_9	API_9Instance17
10	API_9	API_9Instance18
11	API_9	API_9Instance19
12	API_9	API_9Instance20
13	API_9	API_9Instance21
14	API_9	API_9Instance22
15	API_9	API_9Instance23
16	API_9	API_9Instance24
17	API_9	API_9Instance25
18	API_9	API_9Instance26
19	API_9	API_9Instance27

Second Level Resultant Matrix		
ID	Service	Recommendation
1	API_8	NO
2	API_9	OK

Fig. 6 Get service of 3DScan application information in the application server (upper part: operation statistics and lower part: KPI analysis graph)

	INSTANCE_ID	API_ID	MTTR	AVAILABLE_MEMORY	RISK	THROUGHPUT_SEC	MATURITY	RECOMMENDATION	SERVICE_TIME
1	API 8Instance 85	API 8	0.0117	0.1685	0.0101	0.1413	2	NO	0.7522
2	API 8Instance 86	API 8	0.0756	0.4504	0.0117	0.2663	2	NO	0.5071
3	API 8Instance 98	API 8	0.0715	0.7331	0.0062	0.0743	2	MayBe	0.3056
4	API 8Instance 99	API 8	0.0699	0.3665	0.0038	0.0972	2	NO	0.0346
5	API 9Instance 16	API 9	0.018	0.5644	0.015	0.0586	2	NO	0.065
6	API 9Instance 18	API 9	0.0841	0.0284	0.0002	0.0782	2	NO	0.2522
7	API 9Instance 26	API 9	0.0844	0.4945	0.0435	0.1683	2	NO	0.0351
8	API 9Instance 29	API 9	0.0496	0.3069	0.0118	0.2367	2	MayBe	0.2489
9	API 9Instance 30	API 9	0.102	0.4045	0.0043	0.2601	2	NO	0.0134
10	API 9Instance 33	API 9	0.081	0.1953	0.0198	0.071	2	NO	0.6987
11	API 9Instance 37	API 9	0.0068	0.8423	0.0017	0.1983	2	NO	0.1757
12	API 9Instance 39	API 9	0.0691	0.6222	0.0015	0.1582	2	MayBe	0.2132
13	API 9Instance 61	API 9	0.1076	0.8611	0.003	0.2019	2	NO	0.6084
14	API 9Instance 68	API 9	0.0814	0.1999	0.0147	0.2297	2	MayBe	0.4905
15	API 9Instance 74	API 9	0.0644	0.4826	0.0238	0.0202	2	NO	0.3626
16	API 9Instance 85	API 9	0.0025	0.3063	0.0053	0.1677	2	MayBe	0.2637
17	API 9Instance 91	API 9	0.1019	0.0151	0.0456	0.1061	2	NO	0.4593
18	API 9Instance 95	API 9	0.0576	0.7592	0.0084	0.2137	2	NO	0.1243
19	API 10Instance 10	API 10	0.014	0.1988	0.0127	0.147	2	MayBe	0.0593
20	API 10Instance 15	API 10	0.0992	0.0336	0.0009	0.2267	2	NO	0.6903
	INSTANCE_ID	API_ID	PROBABILITY	AVAILABILITY	RESPONSETIME	FAILURE_RATE	DELAY	BANDWIDTH	SEVERITY
1	API 8Instance 85	API 8	0.0908	0.9982	0.3634	0.0305	0.8389	10	0.1111
2	API 8Instance 86	API 8	0.0939	0.9874	0.9144	0.0368	0.0094	10	0.125
3	API 8Instance 98	API 8	0.0185	0.9904	0.6542	0.0018	0.8398	10	0.3333
4	API 8Instance 99	API 8	0.0419	0.9805	0.4597	0.0935	0.7421	10	0.0909
5	API 9Instance 16	API 9	0.0748	0.9904	0.0566	0.097	0.222	10	0.2
6	API 9Instance 18	API 9	0.0012	0.9974	0.7736	0.0513	0.5113	10	0.125
7	API 9Instance 26	API 9	0.0869	0.9891	0.7853	0.0659	0.6836	10	0.5
8	API 9Instance 29	API 9	0.0589	0.9905	0.7781	0.0128	0.3178	10	0.2
9	API 9Instance 30	API 9	0.0214	0.9864	0.5197	0.0782	0.622	10	0.2
10	API 9Instance 33	API 9	0.0794	0.9897	0.3266	0.0128	0.1791	10	0.25
11	API 9Instance 37	API 9	0.0152	0.9984	0.1826	0.0804	0.8664	10	0.1111
12	API 9Instance 39	API 9	0.0121	0.9877	0.2882	0.0114	0.1242	10	0.125
13	API 9Instance 61	API 9	0.0274	0.9878	0.5523	0.0377	0.3603	10	0.1111
14	API 9Instance 68	API 9	0.0734	0.9905	0.5272	0.045	0.2102	10	0.2
15	API 9Instance 74	API 9	0.0238	0.9878	0.3848	0.0843	0.7361	10	1
16	API 9Instance 85	API 9	0.042	0.9834	0.1205	0.0141	0.3783	10	0.125
17	API 9Instance 91	API 9	0.0912	0.9934	0.1453	0.0808	0.1715	10	0.5
18	API 9Instance 95	API 9	0.0587	0.9816	0.9849	0.0533	0.1273	10	0.1429
19	API 10Instance 10	API 10	0.0761	0.9898	0.1415	0.0298	0.7055	10	0.1667
20	API 10Instance 15	API 10	0.0037	0.987	0.4261	0.0866	0.3856	10	0.25

Fig. 7 Analytical results of model generation

MayBe with OK. 1.6251 instances are resulted in the class of NO with MayBe and NO with OK while 0 instances justified for NO with NO class. Class OK with MayBe and class OK with NO justified 147.2743 instances while 0 instances fulfilled the class of OK with OK. From these results, we can see that 0 costs are benefited in the case of the class MayBe with MayBe, OK with OK and NO with NO.

This iteration 2 of model application generates results for all service instances. However, in order to recommend service for reuse, we need consolidated results for each service from all instances. Therefore, we instantiate PODSont and decision-based inference rules presented in Fig. 4 with service instance results of iteration 2 to generate results for each service. The object properties of implemented PODSont are shown in Fig. 8.

The next subsection uncovers different decision scenarios developed for the capitalization of service reuse based on the above performance-oriented data mining analytics and PODSont instantiations.

Table 2 Resultant matrix from decision tree model application

Weight	MayBe	NO	OK
MayBe	0	26,465	26,465
NO	16,251	0	16,251
OK	147,2743	147,2743	0

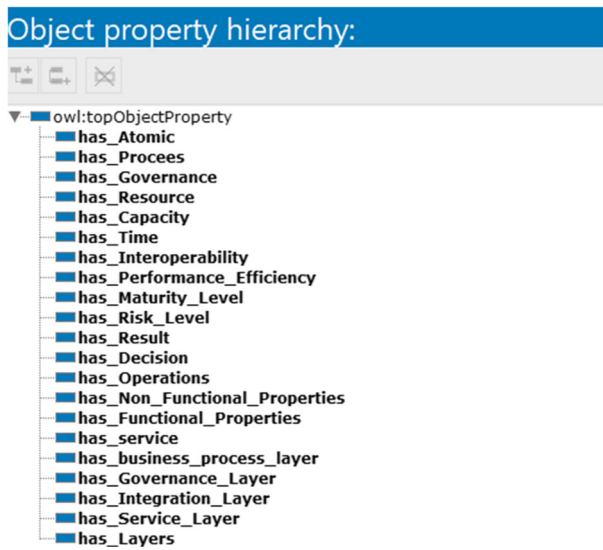


Fig. 8 Object properties of PODSOnt

4.5 Decision scenarios for service reuse

In order to describe the application of classification algorithms, we develop scenarios. A scenario designates a goal for which an End User might use software and all the features of the software that they would require in order to achieve the desired goal. We will provide in detail the unfolding of SPODSS scenarios. We will also present the ability of the system to make a fast and reliable decision.

4.5.1 Scenario 1: Decision results based on the priority of end-user

Specific decisions are made based on the priority of technical indicators specified by End User. These decision results can be provided in both graph and tabular form as desired by End User. We show a graph-based representation for the recommendation of service based on the priority of response time as shown in Fig. 9. This figure shows the response time fluctuations of service instances in different time intervals.

We illustrate through a graph-based representation for the recommendation of service based on the priority of maturity evaluation based on instance ID as shown in Fig. 10. First level evaluation is made based on the maturity for all instances. From this graph, we recommend a service that has less percentage recommendation of NO with the minimum response time. The

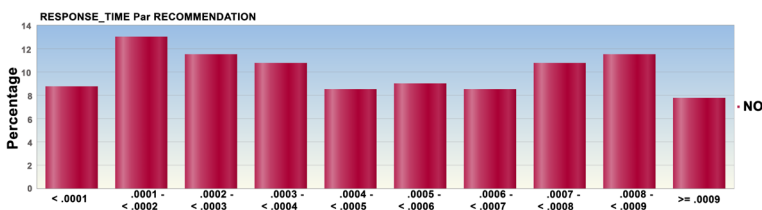


Fig. 9 Response time based on service instanceID

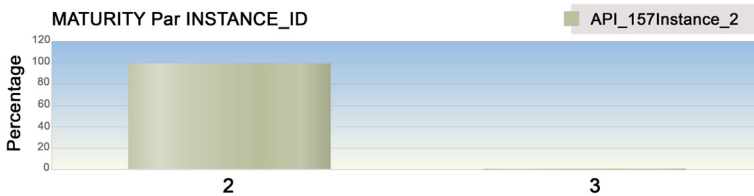


Fig. 10 Maturity evaluation based on service instanceID

horizontal axis shows the response time values that start with =0.0009. The vertical axis shows the percentage of recommendation class NO that starts at 0 and goes until 14.

The light green color service which is named API157 has 100% instances that satisfy maturity level 2 while 1% instances of the same service lie at maturity level 3.

4.5.2 Scenario 2: Decision results for each service

Decision results for each service are made based on the generated results for all instances. For this purpose, ontology and inference rules are instantiated with these first-level results. As a result of this, results are generated for each service. We now present the generated decision result in Fig. 11. This figure shows two-level resultant matrices. The first level resultant matrix shows results for all instances produced by iteration 2 of the model application while the second level resultants matrix illustrates the final results for each service, generated by PODSOnt and decision-based inference rules.

4.6 Recommendations

Developing new services for new or changed requirements increases costs in terms of money and time. Therefore, the proposed system generates different types of possible recommendations in the context of performance-based service reuse. These recommendations are explained below.

4.6.1 Recommendation 1 atomic service

A service task is implemented for atomic service. We recommend atomic services based on the needs of End User. Possible recommendations of atomic services that (i) satisfy functional requirements with performance-based on the priority of technical indicator specified by End User, and (ii) satisfy functional requirements with increased performance based on all technical indicators.

4.6.2 Recommendation 2 composite service

If atomic service is not available or not fulfilling the functional requirements of the user request for service, composite service will be recommended. A composite service is created by combining atomic services together. Possible recommendations of composite services that (i) satisfy functional requirements with performance-based on the priority of technical indicator specified by End User, and (ii) satisfy functional requirements with increased performancebased on all technical indicators.

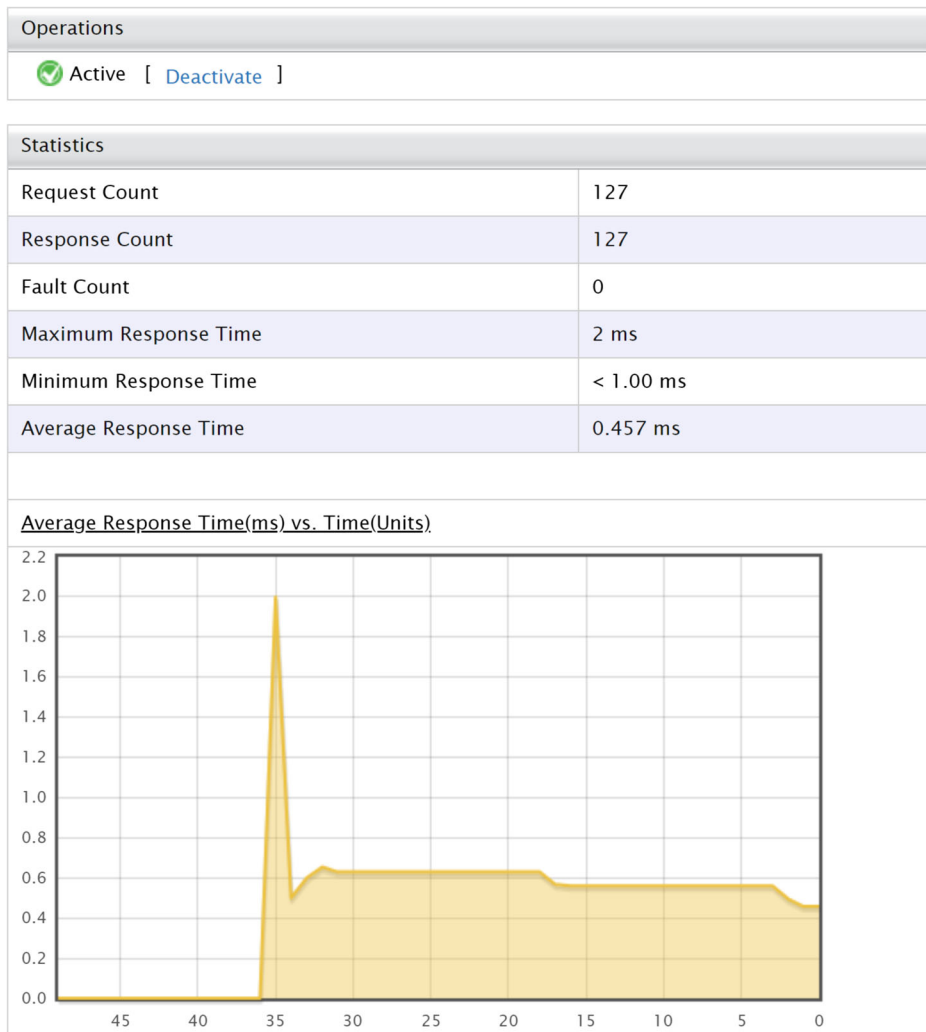


Fig. 11 Decision result for each service

4.6.3 Recommendation 3 resources

Resource allocation is recommended based on dynamic QoS provisioning. It involves the allocation and management of resources at run-time to satisfy certain application QoS requirements. Following actions are suggested to be handled dynamically.

- Actions to be taken when performance fluctuates with the increase in consumption of data.
- Provisioning mechanisms in the infrastructure that need to be measured and controlled dynamically for the triggered event of resource allocation.
- Provisioned resources restoration
- Up-gradation or migration

The results obtained from our proposed SPODSS are encouraging and significant.

5 Conclusions and future research

This research work provides a semantic performance-oriented decision support system called SPODSS for accelerating the performance of SBS by recommending reuse of services to ensure sustainability. For the validation of this research work, a case of application in the SOA industry has been treated in this paper with a network of partners. On the other hand, the case study described, allowed us to illustrate the use of the SPODSS and the real interest of such a tool in the complement of the analysis of the SBS over time. We have seen the contribution of our proposal in the responsiveness of the decision making in terms of the evolution of SBS. Data is classified by classifier while decision generator perform two way validation of results (analytical assessment-traces validation and prediction). As a result, service has been recommended to reuse which is performance efficient, available, mature and incurs less risk. Dynamic evolution of service along its life-cycle is handled by semantic enrichment, and validation of results for the implementation of SPODSS on a service reuse use case. This research work still has some areas to improve for further research. For the moment, we are recommending the reuse of existing services and processes. However, there can be a scenario in which it is necessary to implement a new service with new SLA or implementing a new business process. It is also important to calculate the added value from the economic point of view of the application by using real-time cost analytics data.

Acknowledgements This paper presents work developed in the scope of the project vf-OS. This project has received funding from the European Union Horizon 2020 research and innovation program under grant agreement no. 723710. The content of this paper does not reflect the official opinion of the European Union. Responsibility for the information and views expressed in this paper lies entirely with the authors. This work is done under a Ph.D. program, supervised and defended at the Université Lumière Lyon 2, DISP laboratory and the complete thesis is available on Hal (<https://hal.archives-ouvertes.fr/>).

References

- Adam, F. (2008). *Encyclopedia of decision making and decision support technologies*, volume 2. IGI Global.
- Ahmed-Kristensen, S., & Vianello, G. (2015). A model for reusing service knowledge based on an empirical case. *Research in Engineering Design*, 26(1), 57–76.
- Andrew A Allen, F. M. Costa, and P. J Clarke. A user-centric approach to dynamic adaptation of reusable communication services. *Personal and Ubiquitous Computing*, 20(2):209–227, 2016.
- Arsanjani, A., Zhang, L.-J., Ellis, M. (2007a). Abdul Allam, and Kishore Channabasavaiah. Design an soa solution using a reference architecture. *IBM Developer Works*.
- Arsanjani, A., Zhang, L.-J., Ellis, M., Allam, A., & Channabasavaiah, K. (2007b). S3: A service-oriented reference architecture. *IT Professional*, 9(3), 10–17.
- Asadollah, S. A., and Chiew, T. K. (2011). Web service response time monitoring: architecture and validation. In: *International Conference on Theoretical and Mathematical Foundations of Computer Science*, pages 276–282. Springer.
- Aschoff, R., and Zisman, A. (2011). Qos-driven proactive adaptation of service composition. In: *International Conference on Service-Oriented Computing*, pages 421–435. Springer.
- B OMG. (2008). Business process maturity model (bpmm). *Object Management Group (OMG)*.
- Baghdadi, Y. (2014). Soa maturity models: guidance to realize soa. *International Journal of Computer and Communication Engineering*, 3(5), 372.

- Bani-Ismael, B., and Baghdadi, Y. (2016). Soa maturity models as guidance to select service identification methods: A research agenda. In *2016 IEEE Tenth International Conference on Research Challenges in Information Science (RCIS)*, pages 1–6. IEEE.
- Benaboud, R., Maamri, R., & Sahnoun, Z. (2012). Semantic web service discovery based on agents and ontologies. *International Journal of Innovation, Management and Technology*, 3(4), 467–472.
- Benomrane, S., and Ayed, M. B. (2014). Towards a dynamic knowledge base based on ontology for clinical decision support system. In *2014 6th International Conference of Soft Computing and Pattern Recognition (SoCPar)*, pages 290–293. IEEE.
- Bogner, J., Zimmermann, A., & Wagner, S. (2018). Analyzing the relevance of soa patterns for microservice-based systems. *ZEUS*, 9, 9–16.
- Boumahdi, F., Chalal, R., Guendouz, A., & Gasmia, K. (2016). Soa+d: A new way to design the decision in soa based on the new standard decision model and notation (dmn). *Service Oriented Computing and Applications*, 10(1), 35–53.
- Chhun, S., Moalla, N., & Ouzrout, Y. (2016). Qos ontology for service selection and reuse. *Journal of Intelligent Manufacturing*, 27(1), 187–199.
- Choi, S. W., and Kim, S. D. (2008). A quality model for evaluating reusability of services in soa. In *2008 10th IEEE Conference on E-Commerce Technology and the Fifth IEEE Conference on Enterprise Computing, E-Commerce and E-Services*, pages 293–298. IEEE.
- CMMI Product Team. (2010). Cmmi for services, version 1.3. *Carnegie Mellon University*.
- De Bruin, T., and Doebeli, G. (2015). *An organizational approach to BPM: the experience of an Australian transport provider*, pages 741–759. Springer.
- de Gyves Avila, S., and Djemame, K. (2013). Fuzzy logic based qos optimization mechanism for service composition. In: *2013 IEEE Seventh International Symposium on Service-Oriented System Engineering*, pages 182–191. IEEE.
- D'Mello, D. A., & Ananthanarayana, V. S. (2009). Semantic web service selection based on service provider's business offerings. *International Journal of Simulation: Systems, Science and Technology*, 10(2), 25–37.
- Doultsinou, A., Roy, R., Baxter, D., Gao, J., & Mann, A. (2009). Developing a service knowledge reuse framework for engineering design. *Journal of Engineering Design*, 20(4), 389–411.
- Dream, W. S (2011) Ws-dream datasets. <https://github.com/wsdream/wsdream-dataset>. Accessed 19 Feb 2020.
- Fahad, M., & Qadir, M. A. (2008). A framework for ontology evaluation. *ICCS Supplement*, 354, 149–158.
- Fan, X.-Q. (2013). A decision-making method for personalized composite service. *Expert Systems with Applications*, 40(15), 5804–5810.
- Feuerlicht, G. et al. (2007). Understanding service reusability. In *International Conference Systems Integration*. Department of Information Technologies and Czech Society for Systems Integration.
- Fiware technologies enabling industry 4.0. (2015). <http://www.fiware4industry.com/>. Accessed 19 Feb 2020.
- Giallonardo, E., and Zimeo, E. (2007). More semantics in qos matching. In *IEEE International Conference on Service-Oriented Computing and Applications (SOCA'07)*, pages 163–171. IEEE.
- Gorogiannis, N., Hunter, A., & Williams, M. (2009). An argument-based approach to reasoning with clinical knowledge. *International Journal of Approximate Reasoning*, 51(1), 1–22.
- He, Q., Xie, X., Wang, Y., Ye, D., Chen, F., Jin, H., & Yang, Y. (2016). Localizing runtime anomalies in service-oriented systems. *IEEE Transactions on Services Computing*, 10(1), 94–106.
- Hensle, B., and Deb, M. (2008). Soa maturity model-guiding and accelerating soa success. *Oracle Corporation*.
- Hirschheim, R., Welke, R., and Schwarz, A. (2010). Service-oriented architecture: Myths, realities, and a maturity model. *MIS Quarterly Executive*, 9(1).
- ISO. (2011). Iec25010: 2011 systems and software engineering—systems and software quality requirements and evaluation (square)—system and software quality models. *International Organization for Standardization*, 34, 2910.
- Jakoubi, S., Tjoa, S., Goluch, S., and Kitzler, G. (2010). *Risk-aware business process management—establishing the link between business and security*, pages 109–135. Springer.
- Kahlon, N. K., Kaur, K., and Narang, S. B. (2014). Web services monitoring: A life cycle process. *IUP Journal of Information Technology*, 10(3).
- Khoshkbarforoushha, A., Jamshidi, P., and Shams, F. (2010). A metric for composite service reusability analysis. In *Proceedings of the 2010 ICSE Workshop on Emerging Trends in Software Metrics*, pages 67–74.
- Kohlegger, M., Maier, R., and Thalmann, S. (2009). Understanding maturity models. *Results of a structured content analysis*. na.
- Krivograd, N., Fettke, P., and Loos, P. (2014). Development of an intelligent maturity model-tool for business process management. In *2014 47th Hawaii International Conference on System Sciences*, pages 3878–3887. IEEE.
- Kyusakov, R., Eliasson, J., Delsing, J., Van Deventer, J., & Gustafsson, J. (2012). Integration of wireless sensor and actuator nodes with it infrastructure using service-oriented architecture. *IEEE Transactions on Industrial Informatics*, 9(1), 43–51.

- Lee, J., Lee, D., and Kang, S. (2009). *vPMM: a value based Process Maturity Model*, pages 193–202. Springer.
- Lee, J., Muthig, D., & Naab, M. (2010). A feature-oriented approach for developing reusable product line assets of service-based systems. *Journal of Systems and Software*, 83(7), 1123–1136.
- Lin, L., Kai, S., and Sen, S. (2008). Ontology-based qos-aware support for semantic web services. *Technical Report at Beijing University of Posts and Telecommunications*.
- Masood, T., Cherifi, C. B., and Moalla, N. (2015). Ontology based service network monitoring for better quality of service. In *5th International Conference on Information Society and Technology, In the proceedings of ICIST*, volume 2015, pages 278–283.
- Masood, T., Cherifi, C. B., and Moalla, N. (2016a). Identifying performance objectives to guide service oriented architecture layers. In: *International Conference on Model-Driven Engineering and Software Development*, pages 216–226. Springer.
- Masood, T., Cherifi, C., and Moalla, N. (2016b). Performance monitoring framework for service oriented system lifecycle. In *2016 4th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, pages 800–806. IEEE.
- Masood, T., Cherifi, C. B., Moalla, N., and Fahad, M. (2016c). *Performance Oriented Decision Making to Guide Web Service Lifecycle*, pages 113–122. Springer.
- Masood, T., Cherifi, C., and Moalla, N. (2018). Service recommendation model based on service composition networks monitoring. In *2018 12th International Conference on Software, Knowledge, Information Management & Applications (SKIMA)*, pages 1–8. IEEE.
- McKee, D., Webster, D., and Xu, J. (2015). Enabling decision support for the delivery of real-time services. In *2015 IEEE 16th International Symposium on High Assurance Systems Engineering*, pages 60–67. IEEE.
- Milanovic, N., Milic, B., and Malek, M. (2008). Modeling business process availability. In *2008 IEEE Congress on Services-Part I*, pages 315–321. IEEE.
- Mirandola, R., Potena, P., & Scandurra, P. (2014). Adaptation space exploration for service-oriented applications. *Science of Computer Programming*, 80, 356–384.
- Moraes, P. S., Sampaio, L. N., Monteiro, J. A. S., and Portnoi, M. (2008). Mononto: A domain ontology for network monitoring and recommendation for advanced internet applications users. In *NOMS Workshops 2008-IEEE Network Operations and Management Symposium Workshops*, pages 116–123. IEEE.
- Musen, M. A. (2015). The protege project: a look back and a look forward. *AI Matters*, 1(4), 4–12.
- Oriol, M., Franch, X., and Marco, J. (2010). Salmon: A soa system for monitoring service level agreements. *Universitat Politècnica de Catalunya Technical Report*.
- Oriol, M., Franch, X., & Marco, J. (2015). Monitoring the service-based system lifecycle with salmon. *Expert Systems with Applications*, 42(19), 6507–6521.
- Pakari, S., Kheirkhah, E., & Jalali, M. (2014). A novel approach: A hybrid semantic matchmaker for service discovery in service oriented architecture. *International Journal of Network Security & Its Applications*, 6(1), 37.
- Paschali, M.-E., Ampatzoglou, A., Bibi, S., Chatzigeorgiou, A., & Stamelos, I. (2017). Reusability of open source software across domains: A case study. *Journal of Systems and Software*, 134, 211–227.
- Pereplechikov, M., Ryan, C., and Frampton, K. (2007). Cohesion metrics for predicting maintainability of service-oriented software. In *Seventh International Conference on Quality Software (QSIC 2007)*, pages 328–335. IEEE.
- Pugsley, A. (2006). *Assessing your soa program*. Palo Alto: HP White Pap, Hewlett Packard.
- Pulparambil, S. (2019). *A methodical framework for SOA realization based on SOA maturity model*. PhD thesis, Sultan Qaboos University.
- Pulparambil, S., and Baghdadi, Y. (2015). A comparison framework for soa maturity models. In *2015 IEEE International Conference on Smart City/SocialCom/SustainCom (SmartCity)*, pages 1102–1107. IEEE.
- Pulparambil, S., Baghdadi, Y., Al-Hamdani, A., & Al-Badawi, M. (2017). Exploring the main building blocks of soa method: Soa maturity model perspective. *Service Oriented Computing and Applications*, 11(2), 217–232.
- Rathfelder, C., and Groenda, H. (2008). Isoamm: An independent soa maturity model. In *IFIP International Conference on Distributed Applications and Interoperable Systems*, pages 1–15. Springer.
- Röglinger, M., Pöppelbuß, J., Becker, J. (2012). Maturity models in business process management. *Business Process Management Journal*.
- Rohloff, M. (2009). Process management maturity assessment. *AMCIS 2009 Proceedings*, page 631.
- Sachan, D., Dixit, S. K., & Kumar, S. (2014). Qos aware formalized model for semantic web service selection. *International Journal of Web & Semantic Technology*, 5(4), 83.
- Sackmann, S. (2008). A reference model for process-oriented it risk management. In *ECIS*, pages 1346–1357.
- Sackmann, S., Lowis, L., and Kittel, K. (2009). Selecting services in business process execution-a risk-based approach. *Business Services: Konzepte, Technologien, Anwendungen, Tagung Wirtschaftsinformatik (WI09)*.

- Sadiq, S., Govematori, G., and Namiri, K. (2007). Modeling control objectives for business process compliance. In: *International conference on business process management*, pages 149–164. Springer.
- Sangaiah, A. K., Bian, G.-B., Bozorgi, S. M., Suraki, M. Y., Hosseinabadi, A. A. R., and Shareh, M. B. (2019). A novel quality-of-service-aware web services composition using biogeography-based optimization algorithm. *Soft Computing*, 1–13.
- SCAMPI Upgrade Team. (2011). Standard cmmi appraisal method for process improvement (scampi) a, version 1.3: Method definition document. *Software Engineering Institute, Carnegie Mellon University, Tech. Rep. CMU/SEI-2011-HB-001*.
- Sheng, Q. Z., Qiao, X., Vasilakos, A. V., Szabo, C., Bourne, S., & Xu, X. (2014). Web services composition: A decade’s overview. *Information Sciences*, 280, 218–238.
- Solli-Sæther, H., & Gottschalk, P. (2010). The modeling process for stage models. *Journal of Organizational Computing and Electronic Commerce*, 20(3), 279–293.
- Suriadi, S., Web, B., Winkelmann, A., ter Hofstede, A. H. M., Adams, M., Conforti, R., Fidge, C., La Rosa, M., Ouyang, C., & Pika, A. (2014). Current research in risk-aware business process management—overview, comparison, and gap analysis. *Communications of the Association for Information Systems*, 34(1), 52.
- Tang, Y., and Meersman, R. (2008). Use semantic decision tables to improve meaning evolution support systems. In *International Conference on Ubiquitous Intelligence and Computing*, pages 169–186. Springer.
- Tari, Z., Phan, A. K. A., Jayasinghe, M., and Abhaya, V. G. (2011a). *On the performance of web services*. Springer Science & Business Media.
- Tari, Z., Phan, A. K. A., Jayasinghe, M., and Abhaya, V. G. (2011b). *The Use of Similarity & Multicast Protocols to Improve performance*, pages 59–104. Springer.
- The Open Group. (2011). The open group service integration maturity model (osimm) version 2. <https://www.opengroup.org/soa/source-book/osimmv2/>. Accessed 19 Feb 2020.
- Tizzei, L. P., Nery, M., Segura, V. C. V. B., and Cerqueira R. F. G.. (2017). Using microservices and software product line engineering to support reuse of evolving multi-tenant saas. In *Proceedings of the 21st International Systems and Software Product Line Conference-Volume A*, pages 205–214.
- Tjoa, S., Jakoubi, S., Goluch, G., Kitzler, G., Goluch, S., & Quirchmayr, G. (2010). A formal approach enabling risk-aware business process modeling and simulation. *IEEE Transactions on Services Computing*, 4(2), 153–166.
- Valls, M. G., & Val, P. B. (2013). A real-time perspective of service composition: Key concepts and some contributions. *Journal of Systems Architecture*, 59(10), 1414–1423.
- Valls, M. G., Lopez, I. R., & Villar, L. F. (2012). iLAND: An enhanced middleware for real-time reconfiguration of service oriented distributed real-time systems. *IEEE Transactions on Industrial Informatics*, 9(1), 228–236.
- Wang, F.-J., and Fahmi, F. (2018). Constructing a service software with microservices. In: *2018 IEEE World Congress on Services (SERVICES)*, pages 43–44. IEEE.
- Weber, I., Govematori, G., and Hoffmann, J. (2008). Approximate compliance checking for annotated process models. In: *1st International Workshop on Governance, Risk and Compliance-Applications in Information Systems (GRCISATM08)*.
- Xue, G., Liu, J., Wu, L., & Yao, S. (2018). A graph based technique of process partitioning. *Journal of Web Engineering*, 17(1&2), 121–140.
- Yoon, G., Lee, S., and Choi, H. (2016). Qos optimizer. In *2016 International Conference on Platform Technology and Service (PlatCon)*, pages 1–5. IEEE.
- Zagorulko, Y. A., and Zagorulko, G. (2010). Ontology-based approach to development of the decision support system for oil-and-gas production enterprise. In *SoMeT*, pages 457–466.
- Zheng, Z., Zhang, Y., & Lyu, M. R. (2012). Investigating qos of real-world web services. *IEEE Transactions on Services Computing*, 7(1), 32–39.

Affiliations

Tehreem Masood¹ · Chantal Bonner Cherifi¹ · Nejib Moalla¹

✉ Tehreem Masood
tehreemquireshi@hotmail.com

✉ Chantal Bonner Cherifi
Chantal.BonnerCherifi@univ-lyon2.fr

Nejib Moalla
Nejib.Moalla@univ-lyon2.fr

¹ DISP Laboratory, Universite Lumiere Lyon 2, Lyon, France